

Iterator & Chain of Responsibility

Dois padrões **comportamentais**: o primeiro trata de como percorrer uma coleção sem saber como ela é feita por dentro; o segundo, de como fazer um pedido atravessar uma fila de possíveis tratadores.

Um padrão é o nome de uma solução que já se repetiu o bastante para merecer um. Estes dois se repetem em quase todo sistema.

Onde eles ficam no catálogo

23 PADRÕES · 3 FAMÍLIAS

CRIACIONAIS · 5

Como um objeto nasce.

Factory Method · Abstract Factory · Builder ·
Prototype · Singleton

ESTRUTURAIS · 7

Como objetos se compõem.

Adapter · Bridge · Composite · Decorator · Facade ·
Flyweight · Proxy

COMPORTAMENTAIS · 11

Como objetos **conversam** e dividem responsabilidade.

Chain of Responsibility · Command · **Iterator** ·
Mediator · Memento · Observer · State · Strategy ·
Template Method · Visitor · Interpreter

O que une os dois de hoje: os dois tiram de quem chama uma decisão que ele não deveria estar tomando. O Iterator tira “como eu ando nesta coleção”. O Chain of Responsibility tira “quem é que trata este pedido”.

PARTE UM · PADRÃO COMPORTAMENTAL

01 **Iterator**

Também chamado de *Cursor*. Acessar os elementos de um agregado em sequência, sem expor como ele os guarda.

O problema

Quem percorre uma coleção costuma precisar saber **como ela é feita por dentro**: que é um array, que o índice começa em zero, que o último elemento é o do topo.

Esse conhecimento se espalha. Cada laço novo é mais um lugar que precisa estar certo sobre a mesma coisa — e o dia em que a coleção deixa de ser um array, todos eles quebram ao mesmo tempo.

Três sintomas: a estrutura vaza para quem usa; trocar a estrutura obriga a mexer em todo mundo; e não há como percorrer de duas maneiras diferentes sem duplicar o laço.

```
// quem chama conhece o interior da colecao
for (let i = pilha.itens.length - 1; i >= 0; i--) {
  const id = pilha.itens[i];
  if (!id) continue;
  const item = catalogo.get(id);
  if (!item) continue;
  usar(item);
}

// ... e o mesmo bloco, de novo, em outro lugar
// ... e de novo, com um detalhe diferente
```

O laço acima carrega quatro decisões que não são dele.

“Fornecer um meio de acessar sequencialmente os elementos de um objeto agregado **sem expor sua representação subjacente.**”

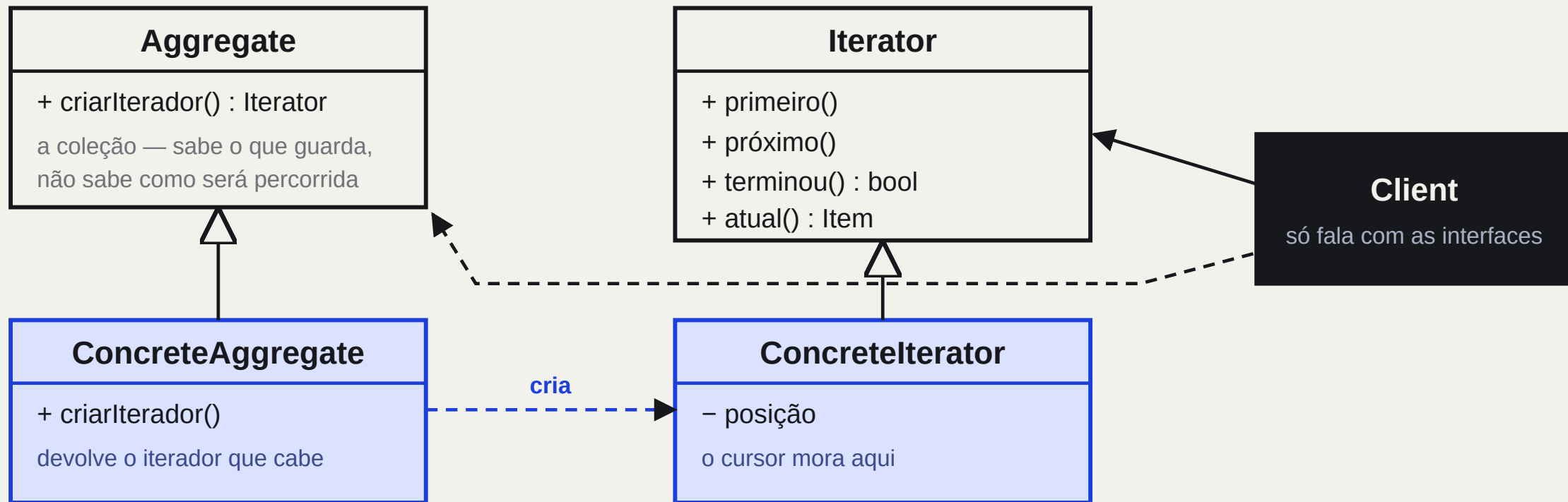
GAMMA, HELM, JOHNSON & VLISSIDES · 1994

A ideia em uma frase

Tirar a responsabilidade de *andar* de dentro da coleção e colocá-la num objeto separado — o iterador — que guarda a posição atual e sabe qual é o próximo.

A consequência imediata

Como a posição vive no iterador, e não na coleção, dá para ter **vários percursos ao mesmo tempo** sobre a mesma coleção — e percursos de tipos diferentes, com a mesma interface.

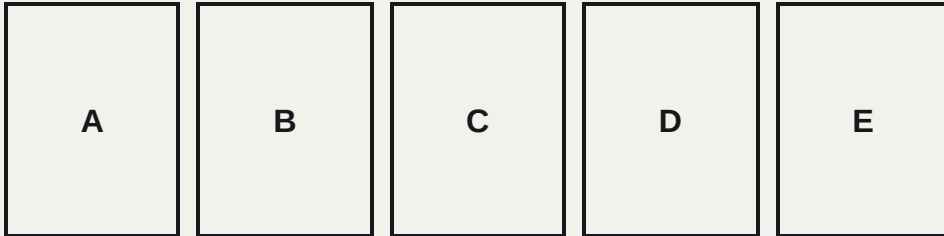


PAPEL	RESPONSABILIDADE	COMO COSTUMA APARECER
Iterator	define a interface do percurso: próximo(), terminou(), atual()	uma interface pequena, quase sempre de 2 a 4 métodos
Concreteliterator	implementa o percurso e guarda a posição atual	uma classe por forma de percorrer: direto, reverso, filtrado, em ordem
Aggregate	declara quem sabe criar um iterador	a interface da coleção
ConcreteAggregate	devolve o iterador concreto adequado à sua estrutura	a lista, a árvore, o mapa — o que realmente guarda os itens
Client	usa o iterador e nunca a estrutura	o laço que ficou com três linhas em vez de dez

Detalhe que separa quem entendeu de quem decorou: a posição fica no *iterador*, não na coleção. É isso que permite dois percursos simultâneos — e é o erro clássico guardar um índice dentro da própria coleção.

Como funciona

A coleção, e o cursor que anda sobre ela



terminou() → false · atual() → A

A coleção não muda enquanto isso. Quem muda é o iterador: ele é o único objeto que sabe **onde parou**.

```
interface Iterador<T> {  
    terminou(): boolean;  
    proximo(): T;  
}  
  
class IteradorDeLista<T> implements Iterador<T> {  
    private posicao = 0;  
    constructor(private itens: T[]) {}  
  
    terminou() { return this.posicao >= this.itens.length; }  
    proximo() { return this.itens[this.posicao++]; }  
}  
  
// o cliente nao sabe o que ha do outro lado  
const it = colecao.iterador();  
while (!it.terminou()) usar(it.proximo());
```

Consequências

ITERATOR · O QUE SE GANHA E O QUE SE PAGA

A FAVOR

- **A estrutura fica escondida.** Trocar array por árvore não mexe em quem percorre.
- **Percursos variados, mesma interface.** Direto, reverso, filtrado, em ordem — o cliente não muda.
- **Vários percursos ao mesmo tempo**, porque cada iterador tem a própria posição.
- **Responsabilidade única.** A coleção guarda; o iterador anda.

CONTRA

- **Mais classes.** Para uma coleção simples, um laço direto é mais honesto.
- **Pode custar desempenho** frente ao acesso direto por índice em coleções muito quentes.
- **Invalidação.** Mudar a coleção durante o percurso é a fonte clássica de bug — defina se o iterador é robusto ou falha rápido.

Quando usar: quando há mais de uma forma de percorrer, quando a estrutura interna pode mudar, ou quando o mesmo laço já apareceu três vezes. **Quando não usar:** quando a coleção é um array local que ninguém mais vê.

Você já usa, todo dia

ITERATOR · NA PLATAFORMA

Java

`Iterable<T>` e `Iterator<T>`. O `for-each` é açúcar sintático em cima deles.

C#

`IEnumerable` / `IEnumerator`, e o `yield return`, que escreve o iterador por você.

Python

`__iter__` e `__next__`. Geradores são iteradores preguiçosos com outra roupa.

JavaScript

`Symbol.iterator`. Todo `for...of`, todo `spread [...x]` é o padrão rodando.

PHP

`Iterator`, `IteratorAggregate` e a biblioteca `SPL` inteira construída em cima.

A lição

Quando um padrão entra na linguagem, ele deixa de parecer padrão. Isso é sinal de que era bom.

PARTE DOIS · PADRÃO COMPORTAMENTAL

02

Chain of Responsibility

Corrente de responsabilidade. Passar o pedido adiante, de tratador em tratador, até que alguém o resolva.

O problema

Um pedido chega e várias verificações precisam acontecer antes de ele ser atendido. A primeira versão é sempre a mesma: uma função que cresce, com um `if` para cada regra.

O incômodo não é o tamanho. É que **a ordem das regras importa** e não está escrita em lugar nenhum; que nenhuma regra pode ser testada sozinha; e que a regra número sete vai ser enfiada no meio das outras seis.

O emissor também sabe demais: ele conhece a lista inteira de quem pode tratar o pedido, e a ordem em que devem ser chamados.

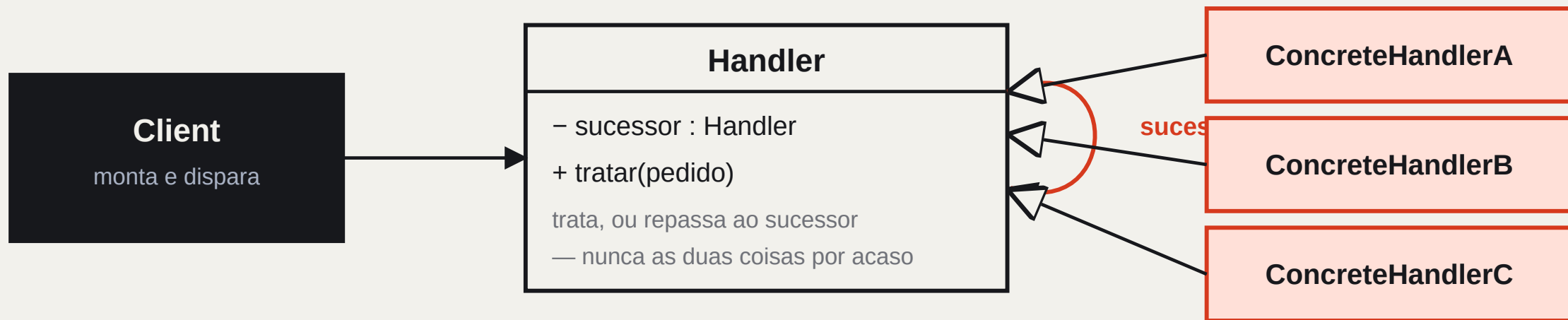
```
function tratar(pedido) {  
  if (excedeuOLimite(pedido)) return recusa('limite');  
  if (!temPermissao(pedido)) return recusa('permissao');  
  if (!formatoValido(pedido)) return recusa('formato');  
  if (estaEmManutencao()) return recusa('manutencao');  
  // ... e a regra nova entra em que linha, mesmo?  
  return executar(pedido);  
}
```

Seis regras, uma função, nenhuma testável isoladamente.

Intenção e estrutura

“Evitar o acoplamento entre o remetente de uma solicitação e seu receptor, dando a **mais de um objeto** a chance de tratar a solicitação. Encadear os objetos receptores e passar a solicitação ao longo da corrente até que um deles a trate.”

GAMMA, HELM, JOHNSON & VLISSIDES · 1994



Como funciona

Pedido: reembolso de R\$ 4.200

Atendente	até R\$ 500
Supervisor	até R\$ 2.000
Gerente	até R\$ 10.000
Diretoria	acima disso
Cada elo faz uma pergunta só: <i>isto é comigo?</i> Se não é, ele repassa. Quem emitiu o pedido não sabe — e não precisa saber — quantos elos existem nem quem vai atender.	

```
abstract class Aprovador {
    private sucessor?: Aprovador;

    seguidoPor(a: Aprovador) { this.sucessor = a; return a; }

    tratar(p: Pedido) {
        if (this.podeAprovar(p)) return this.aprovar(p);
        this.sucessor?.tratar(p);    // repassa
    }

    protected abstract podeAprovar(p: Pedido): boolean;
}

// a ordem vira DADO, e nao aninhamento
atendente.seguidoPor(supervisor).seguidoPor(gerente);
```

Duas variações que valem distinguir

CHAIN OF RESPONSIBILITY · NA PRÁTICA

VARIAÇÃO CLÁSSICA

Para no primeiro que trata

Um elo assume o pedido e a corrente termina ali. É a forma do GoF: aprovação por alçada, tratamento de exceção, resolução de eventos de interface subindo pela árvore de componentes.

Risco: ninguém tratar. A corrente precisa de um elo final que decida o que fazer com o que sobrou.

VARIAÇÃO PIPELINE

Todos passam, cada um faz sua parte

Cada elo verifica ou enriquece e chama o próximo; quem recusa interrompe. É o formato dos *middlewares* e dos filtros de requisição.

Ganho: a ordem vira uma lista visível — e uma regra nova é uma linha nela.

A FAVOR

- Desacopla quem pede de quem atende.
- A ordem vira configuração, não aninhamento.
- Cada elo é testável sozinho.

CONTRA

- Sem elo final, ninguém trata — e ninguém percebe.
- Depurar exige seguir a corrente inteira.
- Corrente longa custa indireção.

Onde ele já está rodando

CHAIN OF RESPONSIBILITY · EXEMPLOS REAIS

Middleware HTTP

Express, Koa, ASP.NET Core. Cada função recebe a requisição e um `next()` — a corrente literal.

Servlet Filters

Java EE. Autenticação, compressão, CORS: cada filtro decide se passa adiante na `FilterChain`.

Logging

Um registro sobe pelos *handlers* por nível: debug, info, erro. Cada um decide se escreve e se repassa.

Eventos de interface

O clique que sobe do botão para o painel e para a janela até alguém tratar — o *event bubbling* do DOM.

Suporte em níveis

$N1 \rightarrow N2 \rightarrow N3$. O exemplo humano do padrão, e o mais fácil de explicar para quem não programa.

O sinal

Se você escreveu `next()`, você já implementou o padrão — com ou sem esse nome.

Os dois, lado a lado

FECHAMENTO

	ITERATOR	CHAIN OF RESPONSIBILITY
Pergunta que responde	como percorro isto sem saber como é feito?	quem trata este pedido?
O que esconde do cliente	a estrutura interna da coleção	a identidade e a quantidade de tratadores
Objeto que ganha vida	o cursor (posição atual)	o elo (decide e repassa)
Relação entre as peças	coleção ↔ iterador, uma de cada vez	elo → elo → elo, em sequência
Como cresce	um iterador novo por forma de percorrer	um elo novo na lista, na posição em que vale
Erro típico	guardar a posição dentro da coleção	nenhum elo tratar, e ninguém perceber

Os dois são, no fundo, a mesma disciplina aplicada a problemas diferentes: **tirar de quem chama uma decisão que não é dele** — e dar a essa decisão um objeto com nome, para que ela possa ser trocada, testada e lida sozinha.

Padrão não é receita: é um nome para uma conversa.

Dizer “aqui cabe um Iterator” economiza dez minutos de explicação — e dizer “isto virou uma corrente de responsabilidade” diz, de uma vez, onde a regra nova deve entrar.

Gamma · Helm · Johnson · Vlissides
*Design Patterns: Elements of Reusable
Object-Oriented Software*, 1994